Development Plan
LingoLyrics

Team #21, LingoLyrics
Rhea Gokhale
Aansh Kotian
Omar Bakr
Raymond Ma

Table 1: Revision History

Date	**Developer(s)**	**Change**
26/09/2026	Aansh Kotian	Draft 0 creation
28/09/2026	Rhea Gokhale	Draft 0 completion and review aligning with problem statement

This document outlines the Development Plan for LingoLyrics, a song-based language-learning product consisting of a mobile learner application and a web-based song-map editor for content creators. It covers project governance, workflow and repository strategy, continuous integration, scheduling, the Proof of Concept (PoC) plan and its risks, technology choices, and the team charter.

1 Confidential Information?

There is no confidential information associated with this project. LingoLyrics is an open-source capstone project operating without corporate non-disclosure agreements, proprietary industry datasets, or commercial restrictions.

2 IP to Protect

There is no proprietary intellectual property (IP) to protect. All application code, file format specifications (song map schema), system documentation, and design assets will be developed openly under standard university capstone guidelines.

3 Copyright License

The software source code, map schema definitions, and associated documentation will be distributed under the **MIT License**. This choice ensures maximum accessibility for community contributors and map creators while permitting open integration with third-party tools.

4 Team Meeting Plan

Meetings will be conducted using a hybrid approach, leveraging specific virtual channels and in-person sessions during scheduled capstone periods.

- **Internal Team Meetings:** Held virtually every Friday from 11:00 AM to 12:00 PM via Discord or Google Meet to review project progress, manage sprint planning, assign issues, and resolve technical blockers.
- **Supervisor Meetings:** Held periodically on Fridays from 10:00 AM to 10:30 AM via Microsoft Teams with our supervising professor, Dr. Angela Zavaleta Bernuy, to present milestone deliverables, track progress, and receive guidance.
- **Meeting Structure:** Meeting operational roles will rotate among team members each session:
  - The meeting chair posts the agenda as a GitHub issue using the meeting issue template at least 24 hours before the meeting.

- Agenda topics are written as questions, each with a time estimate and the member responsible for leading it.
- The first item is always a review of the agenda and meetings last at most one hour.
- Meetings end with a written list of decisions and each member's action items, recorded in the same issue by the minute-taker, along with attendance.

5 Team Communication Plan

Our team will use multiple channels to stay organized and collaborate effectively:

- **Discord**: for quick day-to-day communication, quick queries, and ad-hoc discussions.
- **GitHub Discussions & Issues**: for tracking technical tasks, bug reports, pull request reviews, and project codebase progress.
- **Notion**: for organizing project documents and resources, taking meeting notes, and task planning.

These tools will ensure that all updates, discussions, and decisions are documented and accessible to all team members.

6 Team Member Roles

All team members will be responsible for creating issues, developing, testing, documenting, and reviewing code.

Development work will be partitioned based on project features/components rather than individual technical expertise. For instance, when implementing the end product, one individual will be responsible for the end-to-end development of the song map editor or interactive lesson player components, including frontend UI, backend parsing, and schema integration. This allows technical expertise to be evenly distributed across the team, providing an equal learning and development opportunity for each teammate.

Table 2 lists each member's starting administrative role, the component they own, and the member who backs them up on that component.

Administrative tasks will be broken down into the following operational roles, which will be cycled on a regular basis:

6.1 Project Lead

- Chairs meetings with the team and supervisors by preparing agendas and guiding discussions.

Table 2: Starting Team Roles

Member	Administrative Role	Component Owned	Backup
Omar Bakr	Project Lead	Learner app: playback and lyric sync	Rhea
Raymond Ma	Content and QA Manager	Song-map editor and schema	Aansh
Rhea Gokhale	Minute-Taker	Language processing (segmentation, romanization)	Raymond
Aansh Kotian	DevOps Manager	Anki integration and CI/CD	Omar

- Serves as the primary liaison with the supervisor, Dr. Angela Zavaleta Bernuy.
- Ensures the team is making steady progress toward weekly project goals and capstone milestones.

6.2 Content Manager

- Collects and manages materials needed for external approvals, custom document submissions, and ethics/copyright disclosures.
- Documents external datasets, audio libraries, language processing packages, and academic references utilized throughout the project.

6.3 Minute-Taker

- Takes detailed meeting minutes during internal team meetings and supervisor syncs.
- Highlights upcoming project deliverables, internal task deadlines, and submission requirements.
- Coordinates final document compilation and submits course deliverables.

6.4 DevOps & Resource Manager

- Manages repository configurations, branch protection policies, CI/CD automated pipelines, and cloud web-repository hosting resources.
- Oversees cross-platform compilation environments for desktop (Windows/macOS) and mobile (iOS/Android) targets.

6.5 Flexibility and Rotation

- Administrative roles rotate after each major deliverable: each member moves down one row in Table 2, and the last member moves to the top. The QA role therefore changes hands, so no member reviews their own work every time.
- Each component has a backup member who reviews its pull requests and can take it over if the owner is unavailable or overloaded.
- Every pull request is reviewed by a member who does not own that component, so all members learn every part of the system and can answer questions about it during demonstrations and code review interviews.
- Component ownership is revisited after the PoC demonstration and can be reassigned at a team meeting if workload or interests change; changes are recorded in this document's revision history.

7 Workflow Plan

7.1 Branching

- `main` always contains the latest submitted state of the project. It is protected: changes arrive only through pull requests that pass CI and have at least one approving review.
- `develop` is the integration branch. Feature branches are created from `develop` and merged back into it. We keep this branch so that `main` only changes at submission points.
- Before each deliverable deadline, `develop` is merged into `main` through a release pull request reviewed by the QA manager.
- Each submission is tagged on `main`, for example `PS-DevPlan-Rev0`, `SRS-Rev0`, and `PoC-Demo`. Revision 0 and Revision 1 are distinguished by tags rather than separate files.
- Pull requests are squash-merged and reference their issue with `Closes #N` so issues close automatically with a link to the merge. Work done in pairs uses GitHub co-author commit trailers.

7.2 Labels and Branch Names

Every issue receives one label from Table 3. Branches for code or document changes use the label's prefix and the naming convention `prefix/issue-id-short-description` (for example, `feat/12-sync-player` or `bug/04-anki-payload`).

Table 3: Issue Labels and Git Branch Conventions

Label	**Branch Pattern**	**Scope**
Docs	`docs/*`	Course deliverables, setup guides, and architecture documents.
Feat	`feat/*`	New capabilities in the learner app, editor, or Anki export.
Bug	`bug/*`	Playback timing drift, parsing errors, or runtime exceptions.
Enhancement	`enhance/*`	Refactoring, performance, or UI refinement.
Infra	`infra/*`	CI workflows, build configuration, and dependencies.
Meeting	—	Team meeting agendas, attendance, and minutes.
Lecture	—	Capstone lectures, recorded as team meetings.
Supervisor	—	Supervisor meeting agendas and action items.
Peer-Review	—	Issues received from, or created for, other teams.
Question	—	Open questions needing a team decision or TA input.

7.3 Issues, Milestones, and Pull Requests

- Issues are created from templates: the capTemplate meeting, lecture, and supervisor templates, plus our own feature/bug template that requires a summary, acceptance criteria (the Definition of Done), an owner, a reviewer, and linked dependencies.
- Issues are sized to be completed by one person in a few hours. Large items such as "write SRS" are split into one issue per section.
- Each course deliverable is a GitHub milestone with its due date, and every issue is assigned to a milestone.
- A pull request template asks the author to confirm tests and lint pass, link the issue, and list any AI-assisted changes.
- Deliverable checklists from the course repository are copied into the deliverable's issue and ticked off by the QA manager.

7.4 Continuous Integration

GitHub Actions runs on every pull request and on pushes to `main`:

- build the LaTeX documents to PDF (provided by capTemplate) and publish them to GitHub Pages; generated files are not committed;
- run `ESLint` and `Prettier` checks;
- run the `Jest` unit test suite with a coverage report;
- validate all example song maps against the shared JSON Schema.

A pull request cannot be merged into `develop` or `main` unless all checks pass.

8 Project Decomposition and Scheduling

Task progression and milestone tracking will be managed using GitHub Projects via a dedicated Kanban board to visualize development status. Swimlanes are defined to capture each phase of the task lifecycle:

- **Backlog:** items that have been identified and logged for future iterations
- **To Do:** prioritized tasks scoped for the upcoming sprint, refined by team members
- **In Progress:** active development or documentation tasks underway
- **In Review:** implementation complete, awaiting peer feedback and automated test pass
- **Done:** Definition of Done (DoD) fully satisfied and merged into develop

Major deliverables and target deadlines for the term are outlined below:

- Problem Statement, PoC Plan, and Development Plan – September 28, 2026
- SRS Document and Hazard Analysis – October 19, 2026
- V&V Plan – November 2, 2026
- Design Document Revision -1 & Custom Deliverable Choice – November 16, 2026:
  - System Architecture (Module Guide)
  - Detailed Design (Module Interface Spec)
- Team Contribution Survey and Productivity Report (PoC) – November 20, 2026
- Proof of Concept Demonstration – Between November 23 and December 4, 2026
- Design Document Revision 0 – January 18, 2027:
  - System Architecture (Module Guide)
  - Detailed Design (Module Interface Spec)
- Team Contribution Survey and Productivity Report (Rev 0) – January 29, 2027
- Revision 0 Demonstration – Between February 1 and February 12, 2027
- V&V Report and Custom Deliverable (Winter) – March 8, 2027
- Final Demonstration Revision 1 – Between March 15 and March 22, 2027
- Team Contribution Survey and Productivity Report (Rev 1) – March 19, 2027
- Final Documentation Package – March 31, 2027
- Video Submission & EXPO Poster – TBD / Late March 2027
- Capstone EXPO Demonstration – TBD

Link to GitHub project: https://github.com/rhea333/LingoLyrics/tree/main

9 Proof of Concept Demonstration Plan

The PoC demonstration will show that the three riskiest parts of LingoLyrics can be built with our chosen technology before we commit to full development. It is a narrow technical demonstration, not a prototype of the finished product: it uses one test phone, one short song per language, and no user interface styling.

9.1 Objective

The objective of our PoC is to demonstrate the technical feasibility of real-time audio-lyric synchronization, multi-language text segmentation, and direct flashcard export. The PoC does not need to deliver a fully styled end-to-end consumer application; rather, it should validate that the underlying timing, NLP tokenization, and export workflows operate reliably enough to justify full system implementation.

9.2 Scope of PoC

- **Core Deliverables:** A functional prototype that:
  - Plays back an audio track while maintaining sub-second, real-time highlighting of corresponding lyric lines across desktop and mobile browsers.
  - Accurately parses non-Latin or unspaced text (e.g., Japanese Kanji/Kana or Vietnamese) into interactive words with corresponding transliteration/romanization.
  - Generates and transmits a serialized vocabulary package (including target word, definition, and context sentence) directly into a local or remote Anki instance.
- **Exploratory Nature:**
  1. Evaluating cross-platform web audio timing APIs to achieve reliable sub-second synchronization latency.
  2. Comparing language-specific tokenization pipelines for accurate segmentation of non-spaced scripts.
  3. Testing seamless integration methods with Anki (e.g., AnkiConnect API versus custom file payloads).
- **Not Included:** Complete user account management, custom UI design/styling, broad catalog expansion, or cloud sync backends.

9.3 Risks

Table 4 lists the implementation risks that most threaten the project.

9.4 What We Will Demonstrate

Each item addresses one risk and has a pass criterion.

- **R1:** Play a test song on the test phone with line-by-line highlighting driven by a song map. The app logs the playback position at each line change. *Pass:* at least 95% of line changes occur within 0.5 seconds of their mapped timestamps.

Table 4: PoC Risks

Risk	Why it is a concern
R1: Lyric sync on a phone	Mobile audio playback may not report its position often or accurately enough to keep lyric highlighting within 0.5 seconds of the audio, which the synchronized playback goal requires.
R2: Segmenting lyrics	Tokenizers are built for standard prose; lyrics contain slang, fragments, and mixed scripts. The Japanese dictionary is also large and may load too slowly on a phone.
R3: Anki from a phone	AnkiConnect only works with desktop Anki. It is unclear whether the mobile app can add notes to a learner's Anki collection directly (for example, through the AnkiDroid API) or must use an export file.

- **R2:** Open a Japanese test song and display each lyric line as selectable words with romanization. *Pass:* at least 90% of words in 20 test lines match a segmentation checked by the team, and the lesson is ready within 3 seconds of opening.
- **R3:** Select a word in the app and send it to Anki. *Pass:* a note with the term, translation, and context sentence appears in a test deck, using at least one of the candidate methods.

Not included: user accounts, visual design, the song-map editor interface (maps are written by hand for the PoC), cloud sync, and more than one song per language.

9.5 If the PoC Fails

If a risk is not resolved by the demonstration, we will discuss redefining the scope with our supervisor and the instructor. Likely directions are:

- **R1:** highlight lyrics at a coarser tolerance, or use a native audio module instead of the Expo audio library.
- **R2:** compute segmentation in the editor and store it in the song map, where the creator verifies it, rather than segmenting on the phone.
- **R3:** replace direct sending with an export file that the learner imports into Anki, and revise the vocabulary handoff goal.

9.6 Demonstration Logistics

Each component owner in Table 2 demonstrates the risk for their component. The demonstration runs on one team member's phone, mirrored to a laptop

screen.

10 Expected Technology

Table 5 lists the main technology decisions and the reason for each. All code is written in **TypeScript**, so the learner app, the editor, and the shared song-map types use one language. The development environment is Visual Studio Code.

Table 5: Technology Choices

Tool	**Why chosen**	**Trade-off**
React Native with Expo (learner app)	One codebase for iOS and Android; Expo speeds up builds and testing on real phones.	Less low-level audio control than fully native code (see R1).
React (web editor)	Shares TypeScript types and song-map code with the app; creators need a large screen for timing work.	A second user interface to maintain.
Expo Audio / Howler.js	Audio playback with position reporting in the app and the editor respectively.	Timing accuracy must be verified (R1).
kuromoji.js	Japanese segmentation and readings that run in JavaScript, with no separate server.	Large dictionary (R2); Sudachi is more accurate but needs a Python or Java service.
AnkiDroid API / AnkiConnect / export file	Candidate methods for sending vocabulary to Anki.	Each covers different platforms; the PoC decides (R3).
JSON Schema	Defines and validates the shared song-map format in CI.	The schema must be versioned as it changes.
Jest	Unit and integration tests with built-in coverage.	None significant for our stack.
ESLint, Prettier, Husky	Enforce the coding standard before commit and in CI.	Initial configuration time.
TypeDoc	Generates API documentation from code comments.	Comments must be kept current.

Version control, issue tracking, and scheduling use Git, GitHub, and GitHub Projects; CI runs on GitHub Actions as described in the Workflow Plan.

11 Use of AI and LLMs

- **Code:** AI coding assistants (for example, GitHub Copilot and OpenAI Codex) may generate boilerplate, utility functions, and test cases.
- **Documentation:** LLMs may be used to review drafts against the course checklists and rubrics, suggest wording, and check LaTeX syntax.
- **Verification:** no AI output is merged without human review. AI-generated code must pass lint and tests and is noted in the pull request description; AI-suggested text is checked by the author against our own project decisions and by the QA manager against the checklist.
- **Accountability:** every member must be able to explain any AI-assisted code they submit, since the PoC and final demonstrations include code review interviews.

12 Coding Standard

All code follows the **Airbnb JavaScript/TypeScript Style Guide**, enforced by `ESLint` in Husky pre-commit hooks and in GitHub Actions. Formatting is handled by `Prettier`, and public functions are documented with TSDoc comments.

13 Appendix — Generative AI Use Disclosure

This statement covers both the Problem Statement and Goals and this Development Plan.

13.1 AI Tools Used

OpenAI Codex, using the GPT-5.6 Terra model with medium reasoning effort, was used through the Codex desktop application.

13.2 How and Where Used

Codex was used to review, reword, restructure, and propose candidate wording for the *Problem Statement and Goals* document. In particular, it assisted with clarifying the intended user group; improving the problem description; organizing the inputs, outputs, stakeholders, and environment sections; defining key terms; and presenting the minimum viable product and stretch goals in tables. It also suggested measurable acceptance conditions for the goals and identified document-level issues such as undefined terms, template commentary, and incomplete scope boundaries.

The team used Codex as a drafting and editing aid, not as an authoritative source. The final Problem Statement was edited by the team to reflect its own decisions about the project scope, target users, goals, and evaluation criteria.

This disclosure covers both the Problem Statement and the Development Plan. At the time of this revision, no AI-generated or AI-modified content has been retained in the body of the Development Plan other than this disclosure. This section will be updated if material AI assistance is used in later Development Plan revisions.

13.3 Verification of AI-Generated Content

The team verified AI-assisted material before retaining it by:

- reviewing and editing every AI-assisted passage to ensure it reflects the team's intended deliverable and does not introduce unapproved features or commitments;
- checking that the problem statement, inputs and outputs, stakeholders, environment, goals, and stretch goals use consistent terminology and scope boundaries;
- replacing vague success statements with explicit acceptance criteria that the team can evaluate during implementation and testing;
- checking technical, copyright, licensing, and integration claims against the team's implementation plan, supervisor guidance, and primary sources where external facts are required; and

- conducting a final team review for accuracy, completeness, consistency, grammar, and appropriateness for the capstone audience.

The team remains responsible for the accuracy, completeness, consistency, and appropriateness of all submitted material, whether it was initially written by a team member or assisted by an AI tool.

Appendix — Reflection

1. Why is it important to create a development plan prior to starting the project?

   Writing the plan forced us to make decisions we had been postponing. For example, writing the technology section exposed that half of our drafts assumed a web app while the Problem Statement described a phone app, and writing the PoC plan made us admit that we did not know whether Anki can be reached from a phone at all. Finding these gaps now costs a conversation; finding them in February would cost rewritten code. The plan also gives us agreed rules (branch names, review requirements, charter triggers) that we can point to instead of arguing case by case.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

   Advantages: CI checks every pull request the same way, so lint errors, failing tests, and invalid song maps are caught before they reach `develop`. This matters for us because the editor and the learner app both depend on the same song-map format: a schema check in CI catches a change that would silently break the other component. CI also builds our PDFs automatically, so the submitted documents always match the source.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

   While organizing the Proof of Concept (PoC) plan and overall development strategy, our team had two key technical debates:

   - **Target platform (web vs. mobile):** some members preferred a web application because browser audio APIs are familiar and one codebase would reach every device. Others argued that learners mostly listen to music on their phones with headphones. We resolved this by splitting by user: the learner experience is a mobile app, while the creator editor is web-based, because precise timing work needs a large screen. Both share one song-map format.
   - **Primary audience (serious vs. casual learners):** we disagreed about whether to design for learners who want detailed word breakdowns and Anki export, or for casual listeners. We chose self-directed serious early learners as the primary audience for our goals and PoC, while keeping the interface simple enough for casual use.

Appendix — Team Charter

External Goals

The team's main goal is to create an engaging, accessible tool that bridges music consumption and language acquisition. Through LingoLyrics, we aim to strengthen our technical capabilities in cross-platform web audio engineering, multi-language NLP tokenization, and third-party workflow integrations like Anki. We also hope to deliver a polished, technically rigorous project that stands out at the Capstone EXPO, serves as a strong portfolio piece for full-stack software development, and earns recognition from judges and peers.

Attendance

Expectations

The team expects all members to attend scheduled syncs and arrive on time. If a member is unable to attend or will be late, they are expected to notify the team in Discord ahead of time, review the recorded meeting notes, and provide an async status update on their assigned tasks. Prior to each meeting, the meeting chair will share an agenda outlining key discussion points (e.g., audio engine performance, map editor features, SRS integration). Meetings will end early if all agenda topics are resolved, or continue asynchronously in Discord if further technical discussion is needed.

Acceptable Excuse

An acceptable excuse for missing a meeting is an unpredictable personal emergency, illness, or documented academic exam conflict communicated ahead of time. Missing meetings due to poor time management or avoidable scheduling conflicts is unacceptable.

For project deliverables, acceptable excuses include unforeseen personal emergencies or severe technical blockers that were proactively flagged to the group. Unacceptable excuses include underestimating task complexity, poor time planning, or neglecting assigned components until right before deadline dates. We value transparency and expect everyone to communicate potential delays early so the team can adjust task allocations.

In Case of Emergency

Team members must notify the group via Discord as soon as practically possible during an emergency. For occasional occurrences, team members will redistribute critical tasks (such as urgent PR reviews or deliverable sections) to keep project development moving forward. However, if a member experiences repeated issues meeting deadlines, the team will schedule a check-in to re-balance responsibilities or consult course instructors if necessary.

Accountability and Teamwork

Quality

Deliverables and codebase additions must meet Level 4 standard criteria on project rubrics. Code merged into main branches must pass automated CI linting checks, unit tests, and receive approval from a designated peer reviewer. If team members encounter unfamiliar domain concepts—such as complex foreign text tokenizers or cross-platform web audio APIs—they are expected to research solutions independently and consult team members to maintain high technical standards.

Attitude

Team members must maintain a professional, supportive, and open-minded attitude. Feedback during pull request reviews or planning sessions should remain constructive and focused on code quality and project objectives rather than personal preferences.

Stay on Track

Triggers. Any of the following starts the process below:

- attendance below 90% in a term, or two unexcused absences;
- an assigned task more than 48 hours past its due date without prior notice;
- no commits, reviews, or closed issues for two consecutive weeks;
- missing the internal 48-hour deadline for a deliverable section.

Consequences. Each step applies if the previous one does not resolve the problem within one week:

1. A private check-in with the project lead to understand the cause and agree on a plan.
2. Discussion at a team meeting, recorded in the meeting issue, with specific tasks and dates.
3. The member's tasks are reassigned, and the situation is reflected honestly in the Team Contribution Survey.
4. The issue is raised with our TA, and then with the instructor.

Rewards. After each demonstration, the team goes out for a meal together. A member who closes the most issues in a milestone chooses the restaurant.

Team Building

Our team will hold regular in-person meetings each week to foster strong working relationships. We also plan to dedicate brief informal time before sprint planning sessions and organize monthly team social activities to maintain high team morale and open communication.

Decision Making

Decisions will be made through technical consensus following research and discussion of trade-offs. If the team reaches an impasse on architectural choices (e.g., platform targets or language tokenization libraries), members will present brief technical comparisons, consult with project advisors if needed, and conduct a formal team vote to reach a binding decision.